LexiTAC - Symbol Table, Lexical Analysis & TAC Generation

Submitted By

Student Name	Student ID
Md. Emon Hossain (**Team Leader**)	232-15-818
Mostafizur Rahman Antu	241-15-631
Nahid Al Galib	241-15-577
Mahjabin Sultana	241-15-899

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE 314, Compiler Design Lab

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

19 April 2026

DECLARATION

We hereby declare that this lab project report titled **LexiTAC** — Symbol Table, Lexical Analysis & TAC Generation has been completed by us under the supervision of **Rabeya Khatun, Lecturer,** Department of Computer Science and Engineering, Daffodil International University. We further declare that neither this project nor any part thereof has been submitted elsewhere for academic evaluation.

Submitted To:

Rabeya Khatun
Lecturer
Department of Computer Science and Engineering
Daffodil International University

Submitted by

Md. Emon Hossain (TL) ID:232-15-818 Dept. of CSE, DIU	
Mahjabin Sultana ID: 241-15-899 Dept. of CSE,DIU	Nahid Al Galib ID:241-15-577 Dept. of CSE, DIU
Mostafizur Rahman Antu ID: 241-15-631 Dept. of CSE, DIU	

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:.

Table 1: Course Outcome Statements

CO	Statement
CO1	Understand the working procedure of a compiler and apply it for debugging and analyzing programs.
CO2	Analyze the role of syntax, grammar, and semantics of programming languages in compiler construction.
CO3	Apply industry-standard techniques and tools (Flex/Bison) in designing and implementing compiler phases.
CO4	Create and present a compiler project demonstrating effective communication through documentation and demonstration.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Bloom's Level	Knowledge Profile	CEP
CO1	PO1	C1, C2	KP1–4: Computing fundamentals	EP1: Depth of Knowledge
CO2	PO2	C3, C4	KP1–4, KP6: Analysis & Design	EP1, EP3: Analysis
CO3	PO3	C4, A2	KP6: Tool Application	EP1, EP3: Problem Solving
CO4	PO10	C6, A2, P3	KP4: Communication	EP3, EP5: Stakeholder

The mapping justification of this table is provided in section **4.3.1**, **4.3.2**

Table of Contents

Table of Contents

Declaration.....i
Course & Program Outcome.....ii

Chapter 1: Introduction.....05
1.1 Introduction.....05
1.2 Motivation.....05
1.3 Objectives.....05
1.4 Feasibility Study.....06
1.5 Gap Analysis.....07
1.6 Project Outcome.....07

Chapter 2: System Overview & Architecture.....9
2.1 System Overview.....9
2.2 Supported Grammar.....9
2.3 System Architecture Diagram.....9
2.4 System Components.....10

Chapter 3: Tools & Technologies Used.....11
3.1 Why Flex and Bison?.....11

Chapter 4: Methodology.....12
4.1 Development Methodology.....12
4.2 Overall Project Plan.....12

Chapter 5: Implementation & Result Details.....13
5.1 Lexical Analysis (lexer.l).....13
5.2 Syntax Analysis (parser.y).....16
5.3 TAC Generation.....21
5.4 Standalone C Implementation.....21
5.5 Build Automation (Makefile).....21
5.6 Result and Discussion.....21

Chapter 6: User Interface Design.....23
6.1 Overview.....23
6.2 Design Philosophy.....23
6.3 Interface Panels.....23
6.4 UI Design.....24
6.5 Live Access.....24

Chapter 7: Features of the System.....25

©Daffodil International University

Chapter 8: Challenges Faced & Solutions.....26

Chapter 9: Testing & Results.....27
9.1 Test Cases and Expected Output.....27
9.2 Performance Analysis.....27

Chapter 10: Engineering Standards & Mapping.....28
10.1 Impact on Society.....28
10.2 Project Management.....28
10.3 Complex Engineering Problem Mapping.....29
10.4 Program Outcome Mapping.....29

Chapter 11: Conclusion.....30
11.1 Summary.....30
11.2 Limitations.....30
11.3 Future Work.....30

References.....30

©Daffodil International University

Chapter 1

Introduction

This chapter introduces the fundamental concepts, motivations, and objectives behind the LexiTAC compiler front-end project.

1.1 Introduction

A compiler is a complex software system responsible for translating high-level source code into an equivalent lower-level representation. The process is divided into several well-defined phases, each transforming the program closer to executable machine code. Among these phases, the front-end — comprising lexical analysis, syntax analysis, and intermediate code generation — is foundational to understanding compiler theory and practice.

LexiTAC is a modular compiler front-end built entirely from scratch using Flex (for lexical analysis), Bison (for LALR(1) syntax analysis and TAC generation), and C (for supporting code and the standalone implementation). The project uniquely extends the traditional CLI-based compiler demo with a fully interactive web-based simulator, making it an exceptional educational and portfolio tool.

1.2 Motivation

The primary motivation for building LexiTAC stemmed from the observation that most compiler demo tools available for students are either too simplistic to demonstrate real compilation concepts, or too complex and monolithic to be understood by beginners. Specifically, students in Compiler Design courses often struggle with:

- Understanding how raw source text is decomposed into meaningful tokens by a lexical analyzer.
- Appreciating the role of the symbol table in tracking variable identities and types.
- Visualizing how an LALR(1) parser enforces grammar rules and builds a parse tree.
- Grasping how Three-Address Code (TAC) is emitted from a parse tree to serve as an intermediate representation.

The addition of a web-based interactive simulator further bridges the gap between theory and practice — a feature not found in most comparable academic projects.

1.3 Objectives

The main objectives of this project are as follows:

1. Design and implement a Flex-based lexical analyzer that tokenizes arithmetic expressions and registers identifiers in a symbol table with auto-generated unique IDs.
2. Implement a Bison-based LALR(1) parser that validates input against a well-defined grammar and correctly handles operator precedence and associativity.
3. Integrate TAC generation as semantic actions within Bison grammar rules, using temporary variables (t1, t2, …) to represent intermediate computations.

©Daffodil International University

4. Maintain a dynamic symbol table that maps all encountered identifiers to unique internal IDs and type annotations.
5. Develop an interactive browser-based web simulator that visualizes the Lexer → Symbol Table → Parser → TAC pipeline in real time with a polished dark-themed UI.
6. Provide a standalone C implementation (leci.c) that functions without requiring Flex or Bison, enabling quick testing on any C compiler.
7. Automate the build process using a Makefile for reproducible compilation.

1.4 Feasibility Study

Technical Feasibility

Flex and Bison are mature, industry-standard tools with extensive documentation and broad platform support (Linux, macOS, Windows via MSYS2). The C11 standard and GCC toolchain required are freely available. The web simulator requires only a modern browser and uses no external dependencies beyond standard HTML/CSS/JavaScript, making it completely portable and zero-install.

Economic Feasibility

The entire project is zero-cost. All tools used — Flex, Bison, GCC, Make — are open-source. The web simulator is deployed for free on Netlify. No proprietary software or paid services are required at any stage.

Operational Feasibility

The terminal-based tool can be operated with a single command after build. The web simulator is accessible via any browser with no installation. Both interfaces are designed with simplicity and user experience in mind, ensuring accessibility for students and evaluators alike.

©Daffodil International University

1.5 Gap Analysis

Requirement	Observed Gap in Existing Tools	Our Solution
Interactive Visualization	CLI-only tools with no visual pipeline	4-panel real-time web simulator
Industry-Grade Tools	Many demos use hand-written tokenizers	Uses Flex + Bison (as used in GCC)
Symbol Table Output	Typically absent or undisplayed	Full tabular output: Name, ID, Type
Operator Precedence	Often handled incorrectly	Declared via %left/%right in Bison
Unary Minus Support	Rare in student projects	Implemented with %prec UMINUS
Portable Fallback	Single implementation only	Standalone C version (no Flex/Bison)
Live Deployment	Lab-only, not publicly accessible	Deployed live at lexitac.netlify.app

1.6 Project Outcome

At the conclusion of this project, the team successfully delivered a complete, multi-implementation compiler front-end that: tokenizes single-line arithmetic assignment expressions; builds and displays a symbol table with auto-generated IDs; validates input against a formal grammar with correct precedence and associativity; and generates clean Three-Address Code. The interactive web simulator provides a live, browser-accessible demonstration of all compiler phases simultaneously.

©Daffodil International University

CHAPTER 2

SYSTEM OVERVIEW & ARCHITECTURE

2.1 System Overview

LexiTAC implements the first three phases of a compiler pipeline, treating the output of each phase as the input to the next. The system accepts a single-line arithmetic assignment expression (e.g., p = i + r * 60) and produces three structured outputs: a list of classified tokens, a populated symbol table, and a sequence of Three-Address Code instructions. This pipeline mirrors the behavior of real-world compiler front-ends, making it an ideal pedagogical tool.

2.2 Supported Grammar

The formal grammar recognized by LexiTAC is defined as follows:

```
stmt   →  IDENTIFIER '=' expr
expr   →  expr '+' term  |  expr '-' term  |  term
term   →  term '*' factor  |  term '/' factor  |  factor
factor →  '(' expr ')'  |  NUMBER  |  IDENTIFIER  |  '-' factor
```

Operator precedence is enforced via Bison's %left and %right directives: addition and subtraction are left-associative at lower precedence; multiplication and division are left-associative at higher precedence; unary minus is declared with %right and %prec UMINUS at the highest precedence.

2.3 System Architecture Diagram

The architecture of LexiTAC follows a classic linear compiler pipeline, as illustrated below:

[Source Expression Input]
↓
[Flex Lexical Analyzer (lexer.l)]
Regex patterns → Classified token stream | Builds Symbol Table
↓
[Symbol Table Manager]
Identifiers → id1, id2, … with type tracking
↓
[Bison LALR(1) Parser (parser.y)]
Grammar rules with semantic actions → Parse tree / AST
↓
[TAC Generator]
Semantic actions emit: t1 = arg1 op arg2
↓
[Three-Address Code Output]

Figure 1: LexiTAC System Architecture Diagram

©Daffodil International University

2.4 System Components

Component	File	Responsibility
Flex Lexer	lexer.l	Tokenizes input using regex; registers identifiers in symbol table; emits token prints
Bison Parser	parser.y	Validates grammar using LALR(1) shift-reduce algorithm; drives TAC emission via semantic actions
Symbol Table Manager	lexer.l (embedded)	Stores identifiers with auto-assigned IDs (id1, id2…) and type (int); supports lookup and deduplication
TAC Generator	parser.y (semantic actions)	Creates temporary variables (t1, t2…); emits binary and unary TAC instructions in correct evaluation order
Web Simulator	index.html	Full-stack browser UI visualizing all 5 compiler phases with color-coded token chips, animated pipeline, and live output
Standalone C Impl.	c_code/leci.c	Self-contained version without Flex/Bison dependency; uses hand-written recursive descent parser
Build Automation	Makefile	GNU Make rules for clean, reproducible multi-step build (Bison → Flex → GCC link)

©Daffodil International University

CHAPTER 3

TOOLS & TECHNOLOGIES USED

3.1 Why Flex and Bison?

Flex and Bison are the direct descendants of the original Lex and Yacc tools that have powered real-world compilers (including early versions of GCC) for decades. Using these tools in a university project provides students with genuine exposure to production compiler-building workflows. Unlike hand-written tokenizers or recursive-descent parsers, Flex and Bison enforce a clean separation between the specification (what tokens/grammar look like) and the implementation (how they are recognized), which is a critical software engineering discipline.

Technology	Version	Role in Project
C (C11 Standard)	C11	Core implementation language for both Flex/Bison generated code and standalone version
Flex (Fast Lex)	2.6+	Lexical analyzer generator — converts regex patterns in lexer.l to a DFA-based tokenizer
Bison (GNU Bison)	3.8+	LALR(1) parser generator — converts grammar rules in parser.y to a shift-reduce parser with semantic actions
GCC	11+	Compiles the generated C files (lex.yy.c + parser.tab.c) into the final executable
GNU Make	Current	Automates 3-step build: Bison generates header → Flex generates scanner → GCC links all
HTML / CSS / JS	ES6+	Powers the interactive web simulator with no build step or external dependencies
JetBrains Mono	Font	Monospace font used in web UI for token and TAC display — improves code readability
Netlify	Cloud	Free static hosting for the live web simulator at lexitac.netlify.app
Git / GitHub	Current	Version control and public repository hosting; enables collaboration and open contribution

CHAPTER 4

METHODOLOGY

4.1 Development Methodology

LexiTAC was developed following an iterative, phase-by-phase construction methodology that mirrors the structure of the compiler itself. Each phase was developed, tested, and integrated independently before moving to the next.

Phase	Activity	Output
Phase 1	Lexer Design (lexer.l)	Working Flex tokenizer that recognizes IDs, numbers, and operators; symbol table structure defined
Phase 2	Symbol Table Implementation	sym_add(), sym_index(), print_symtab() functions tested with isolated input
Phase 3	Grammar Design (parser.y)	LALR(1) grammar with correct precedence; Bison generates conflict-free parser tables
Phase 4	TAC Generation	Semantic actions in grammar rules emit correct TAC; make_tmp() and emit() helpers verified
Phase 5	Integration & Makefile	Unified build; all phases run end-to-end on test expressions
Phase 6	Standalone C Version	leci.c implemented with recursive descent; no Flex/Bison dependency
Phase 7	Web Simulator (index.html)	Full 4-panel interactive UI; JavaScript re-implementation of all phases for browser execution
Phase 8	Testing & Deployment	All test cases verified; live deployment to Netlify; GitHub repository published

4.2 Overall Project Plan

- Week 1: Literature review on compiler theory, Flex/Bison syntax, and TAC generation strategies.
- Week 2: Implementation of lexer.l, symbol table, and initial Bison grammar skeleton.
- Week 3: TAC semantic actions, Makefile, and integration testing.
- Week 4: Standalone C version, web simulator development, and live deployment.
- Week 5: Final testing, report writing, and repository documentation.

©Daffodil International University

CHAPTER 5

IMPLEMENTATION & RESULT DETAILS

5.1 Lexical Analysis (lexer.l)

The Flex specification file lexer.l defines all token patterns using regular expressions. The lexer recognizes the following token classes:

Token Class	Regex Pattern	Action
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	Calls sym_add() to register variable; passes internal ID (id1, id2…) to parser via yylval
NUMBER (Integer)	[0-9]+	Passes literal string to parser via yylval.str; prints <NUMBER:n>
NUMBER (Float)	[0-9]+\.[0-9]+	Passes float literal; same handling as integer numbers
ASSIGN	=	Returns ASSIGN token; prints <=>
PLUS / MINUS	+ / -	Returns PLUS/MINUS; prints <+> or <->
MUL / DIV	* / /	Returns MUL/DIV; prints <*> or </>
LPAREN / RPAREN	(/)	Returns LPAREN/RPAREN; prints <(> or <)>
Whitespace	[\t\r]+	Silently skipped — no token returned
NEWLINE	\n	Returns NEWLINE — acts as statement terminator

The symbol table is implemented as an array of 256 Symbol structs, each containing a name[64], id[8], and type[8] field. The sym_add() function first checks for an existing entry via sym_index(); if the identifier is new, it inserts it and assigns the next sequential ID (id1, id2, …).

Lexer.l

```
%{
/*
 * lexer.l — Flex Lexical Analyzer
 * Companion to parser.y (Bison)
 *
 * Compile:
 *   flex lexer.l
 *   bison -d parser.y
 *   gcc lex.yy.c parser.tab.c -o compiler -lfl
```

```
 *   echo "p = i + r * 60" | ./compiler
 */

#include "parser.tab.h"   /* generated by bison -d parser.y */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ---------------------------------------------------------------- */
/*  Symbol Table                                                    */
/* ---------------------------------------------------------------- */
typedef struct {
    char name[64];
    char id[8];
    char type[8];
} Symbol;

Symbol symtab[256];
int    symcount = 0;

/* Return index of existing symbol, or -1 */
int sym_index(const char *name) {
    for (int i = 0; i < symcount; i++)
        if (strcmp(symtab[i].name, name) == 0) return i;
    return -1;
}

/* Add symbol if new; return index */
int sym_add(const char *name, const char *type) {
    int idx = sym_index(name);
    if (idx != -1) return idx;
    strncpy(symtab[symcount].name, name, 63);
    snprintf(symtab[symcount].id, 8, "id%d", symcount + 1);
    strncpy(symtab[symcount].type, type, 7);
    return symcount++;
}

/* Print symbol table */
void print_symtab(void) {
    printf("\n--- Symbol Table ---\n");
    printf("%-15s %-8s %-8s\n", "Name", "ID", "Type");
    printf("%-15s %-8s %-8s\n", "----", "--", "----");
    for (int i = 0; i < symcount; i++)
        printf("%-15s %-8s %-8s\n",
               symtab[i].name, symtab[i].id, symtab[i].type);
```

```
}
%}

/* Flex options */
%option noyywrap
%option yylineno

/* ---------------------------------------------------------------- */
/*  Token patterns                                                  */
/* ---------------------------------------------------------------- */
DIGIT    [0-9]
LETTER   [a-zA-Z_]
INT      {DIGIT}+
FLOAT    {DIGIT}+"."{DIGIT}+
ID       {LETTER}({LETTER}|{DIGIT})*
WS       [ \t\r]+

%%

{WS}        { /* skip whitespace */ }

{FLOAT}     {
            yylval.str = strdup(yytext);
            printf("<NUMBER:%s> ", yytext);
            return NUMBER;
        }

{INT}       {
            yylval.str = strdup(yytext);
            printf("<NUMBER:%s> ", yytext);
            return NUMBER;
        }

{ID}        {
            int idx = sym_add(yytext, "int");
            yylval.str = strdup(symtab[idx].id);  /* pass id (e.g. id1) */
            printf("<%s> ", symtab[idx].id);
            return IDENTIFIER;
        }

"="         { printf("<=> "); return ASSIGN; }
"+"         { printf("<+> ");  return PLUS;   }
"-"         { printf("<-> ");  return MINUS;  }
"*"         { printf("<*> ");  return MUL;    }
"/"         { printf("</> ");  return DIV;    }
```

©Daffodil International University

```
"("        { printf("<(> ");  return LPAREN; }
")"        { printf("<)> ");  return RPAREN; }

"\n"       { return NEWLINE; }

.          {
              fprintf(stderr, "Lexical Error: unknown character '%s'\n", yytext);
           }

%%
```

5.2 Syntax Analysis — Bison LALR (1) Parser (parser.y)

The Bison specification defines the complete grammar for LexiTAC. The grammar uses four non-terminals: program, stmt, expr, term, and factor. Operator precedence is declared explicitly:

%left PLUS MINUS	/* Lower precedence, left-associative */
%left MUL DIV	/* Higher precedence, left-associative */
%right UMINUS	/* Highest precedence, for unary minus */

The %union directive declares yylval as a union containing a char* str field, which is used to pass identifier IDs and literal values up the parse tree as semantic values. TAC temporary variables are created by the make_tmp() helper function using a global counter (tmpcnt), and instructions are printed by the emit() function.

<u>Parse.y</u>

```
%{
/*
 * parser.y — Bison Parser + Three-Address Code (TAC) Generator
 *
 * Grammar (simplified):
 *   stmt   → IDENTIFIER '=' expr
 *   expr   → expr '+' term | expr '-' term | term
 *   term   → term '*' factor | term '/' factor | factor
 *   factor → '(' expr ')' | NUMBER | IDENTIFIER
 *
 * Compile & Run:
 *   flex  lexer.l
 *   bison -d parser.y
 *   gcc   lex.yy.c parser.tab.c -o compiler -lfl
 *   echo "p = i + r * 60" | ./compiler
 */
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ---------------------------------------------------------------- */
/*  Declarations from lexer.l                                       */
/* ---------------------------------------------------------------- */
extern int   yylex(void);
extern int   yylineno;
extern char *yytext;
void yyerror(const char *msg);

/* Symbol table — defined in lexer.l */
typedef struct { char name[64]; char id[8]; char type[8]; } Symbol;
extern Symbol symtab[];
extern int    symcount;
extern int    sym_index(const char *);
extern int    sym_add(const char *, const char *);
extern void   print_symtab(void);

/* ---------------------------------------------------------------- */
/*  TAC helpers                                                     */
/* ---------------------------------------------------------------- */
static int tmpcnt = 0;

static char *make_tmp(void) {
    char *t = malloc(8);
    snprintf(t, 8, "t%d", ++tmpcnt);
    return t;
}

static void emit(const char *result, const char *arg1,
                 const char *op,    const char *arg2) {
    if (op && arg2)
        printf("%s = %s %s %s\n", result, arg1, op, arg2);
    else
        printf("%s = %s\n",       result, arg1);
}
%}

/* ---------------------------------------------------------------- */
/*  Value type for semantic actions                                 */
/* ---------------------------------------------------------------- */
%union {
    char *str;   /* holds identifier id-string or literal value */
```

```
}

/* ---------------------------------------------------------------- */
/*  Token declarations                                              */
/* ---------------------------------------------------------------- */
%token <str> IDENTIFIER NUMBER
%token ASSIGN PLUS MINUS MUL DIV LPAREN RPAREN NEWLINE

/* ---------------------------------------------------------------- */
/*  Non-terminal types                                              */
/* ---------------------------------------------------------------- */
%type <str> expr term factor

/* ---------------------------------------------------------------- */
/*  Operator precedence (low → high)                                */
/* ---------------------------------------------------------------- */
%left  PLUS MINUS
%left  MUL  DIV
%right UMINUS

%%
/* ================================================================ */
/*  Grammar rules                                                   */
/* ================================================================ */

program
   : /* empty */
   | program stmt
   ;

stmt
   : IDENTIFIER ASSIGN expr NEWLINE
      {
         /* Find the target variable id */
         int idx = sym_index($1);          /* $1 is already the id string  */
         /* Walk symtab to find by id */
         char lhs[8];
         strcpy(lhs, $1);                  /* $1 is id string from lexer   */
         emit(lhs, $3, NULL, NULL);
         free($1); free($3);
      }
   | error NEWLINE { yyerrok; }
   ;

expr
```

©Daffodil International University

```
    : expr PLUS term
        {
            char *t = make_tmp();
            emit(t, $1, "+", $3);
            free($1); free($3);
            $$ = t;
        }
    | expr MINUS term
        {
            char *t = make_tmp();
            emit(t, $1, "-", $3);
            free($1); free($3);
            $$ = t;
        }
    | term  { $$ = $1; }
    ;

term
    : term MUL factor
        {
            char *t = make_tmp();
            emit(t, $1, "*", $3);
            free($1); free($3);
            $$ = t;
        }
    | term DIV factor
        {
            char *t = make_tmp();
            emit(t, $1, "/", $3);
            free($1); free($3);
            $$ = t;
        }
    | factor  { $$ = $1; }
    ;

factor
    : LPAREN expr RPAREN  { $$ = $2; }
    | NUMBER              { $$ = $1; }
    | IDENTIFIER          { $$ = $1; }
    | MINUS factor %prec UMINUS
        {
            char *t = make_tmp();
            emit(t, "-", $2, NULL);   /* unary minus as: t = - x */
            free($2);
            $$ = t;
```

```
        }
    ;

%%
/* ================================================================== */
/*  Supporting C code                                                 */
/* ================================================================== */

void yyerror(const char *msg) {
    fprintf(stderr, "\nParse Error (line %d): %s\n", yylineno, msg);
}

int main(void) {
    char buffer[512];


        printf("==================================================================
        ==================\n");
    printf("        Flex + Bison: Symbol Table, Lexical Analysis & TAC Generator\n");

        printf("==================================================================
        ==================\n");
    printf("Enter expression(s). One per line. Ctrl-D to finish.\n");
    printf("Example: p = i + r * 60\n\n");

    /* ---- Phase 1 header ---- */
    printf("--- Lexical Tokens ---\n");

    yyparse();          /* drives yylex(); TAC emitted inside rules */

    /* ---- Symbol table (populated during lex) ---- */
    print_symtab();

    printf("\n(TAC emitted above during parsing.)\n");

        printf("==================================================================
        ==================\n");
    return 0;
}
```

5.3 TAC Generation — Semantic Actions

TAC is generated inline with parsing via Bison semantic actions enclosed in { } after each grammar rule. The evaluation order is bottom-up (as in all LALR parsers), ensuring that sub-expressions are computed before they are used. For the expression p = i + r * 60, the generated TAC is:

```
t1 = id3 * 60
t2 = id2 + t1
id1 = t2
```

Unary minus is handled as a special grammar rule with the %prec UMINUS directive, emitting a single-operand instruction: t = - x. Parenthesized expressions are handled transparently — the factor rule for LPAREN expr RPAREN simply passes the inner expression's value up without emitting any TAC.

5.4 Standalone C Implementation (c_code/leci.c)

For environments without Flex or Bison, a self-contained implementation is provided in c_code/leci.c. This version uses a hand-written next_token() function for lexical analysis and a recursive descent parser (expr(), term(), factor() functions) for syntax-directed TAC generation. It supports the same grammar and produces identical output to the Flex/Bison version. The standalone binary leci.exe is pre-compiled for Windows.

5.5 Build Automation (Makefile)

The Makefile encodes the three-step dependency chain required by Flex and Bison:

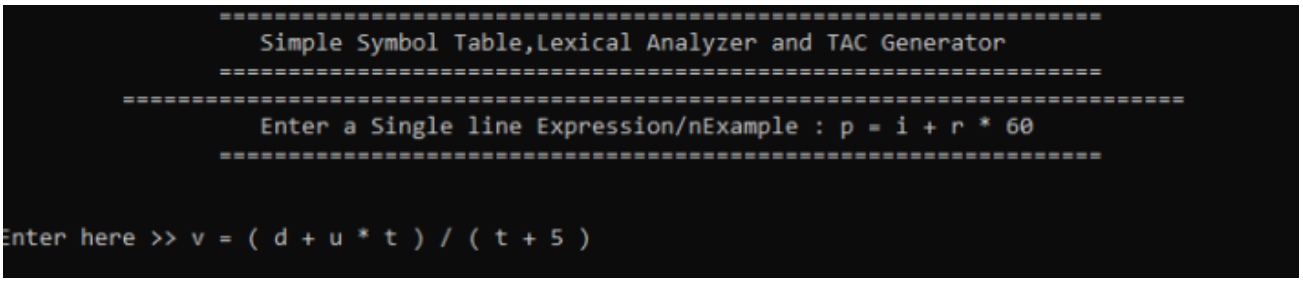
Step 1: bison -d parser.y → Generates parser.tab.c + parser.tab.h
Step 2: flex lexer.l → Generates lex.yy.c (requires parser.tab.h)
Step 3: gcc lex.yy.c parser.tab.c -o compiler -lfl

Targets: make (build) | make run (test) | make clean (remove generated files)

5.6 Result and Discussion

Sample Input:

v = (d + u * t) / (t + 5)



Symbol Table Output:

```
--- Symbol Table ---
v      -> id1
d      -> id2
u      -> id3
t      -> id4
```

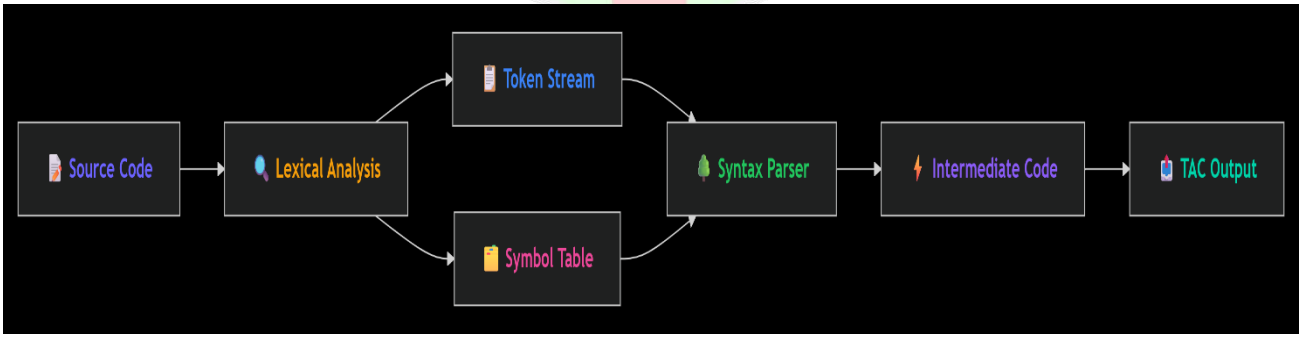
Lexical Tokens:

```
--- Lexical Analysis ---
<id1> <=> <(> <id2> <+> <id3> <*> <id4> <)> </> <(> <id4> <+> <5> <)>
```

TAC:

```
--- Three Address Code (TAC) ---
t1 = id3 * id4
t2 = id2 + t1
t3 = id4 + 5
t4 = t2 / t3
id1 = t4
```

Architecture: Compiler Pipeline



©Daffodil International University

CHAPTER 6

USER INTERFACE DESIGN

6.1 Overview

A distinguishing feature of LexiTAC compared to typical compiler lab projects is its fully interactive, browser-based web simulator. The simulator provides a real-time, four-panel visualization of the entire compiler front-end pipeline, allowing users to type or paste any arithmetic assignment expression and immediately observe the results of all compilation phases simultaneously — without installing any software.

6.2 Design Philosophy

The UI was designed with the following principles:

- Dark-themed, IDE-inspired aesthetic using CSS custom properties for consistent theming across all panels.
- Monospace font (JetBrains Mono) for all code output — tokens, TAC, symbol table IDs — to improve readability and authenticity.
- Color-coded token chips that visually distinguish identifiers (blue), assignment operators (orange), arithmetic operators (green), and numeric literals (teal).
- Animated compiler pipeline panel that highlights each phase step-by-step as the expression is processed.
- Pre-loaded quick examples (e.g., p = i + r * 60, z = (a+b)*(c-d)) for instant demonstration without typing.

6.3 Interface Panels

Panel	Description
Source Input	Full-width code editor panel on the left. User types the source expression here. Features a monospace editor area, placeholder text, and quick-example buttons at the bottom.
Lexical Tokens	Top-right panel displaying color-coded token chips. Each chip shows the token type label and its value (e.g., ID id1, ASSIGN =, NUM 60). Updates in real time on each run.
Symbol Table	Mid-right panel showing a formatted table with columns: #, Name, Internal ID, Type. Populated by the JavaScript symbol table logic, mirroring the C implementation.
Three-Address Code	Bottom-left panel (right side) showing line-numbered TAC output with syntax highlighting. Temporaries (t1, t2) and identifiers (id1, id2) are visually distinguished.
Compiler Pipeline	Bottom-right animated panel with 5 numbered steps: Source Input → Lexical Analysis (Flex) → Symbol Table → Syntax Analysis (Bison) → TAC Generation. Each step has a description and highlight animation.

©Daffodil International University

6.4 UI Design:

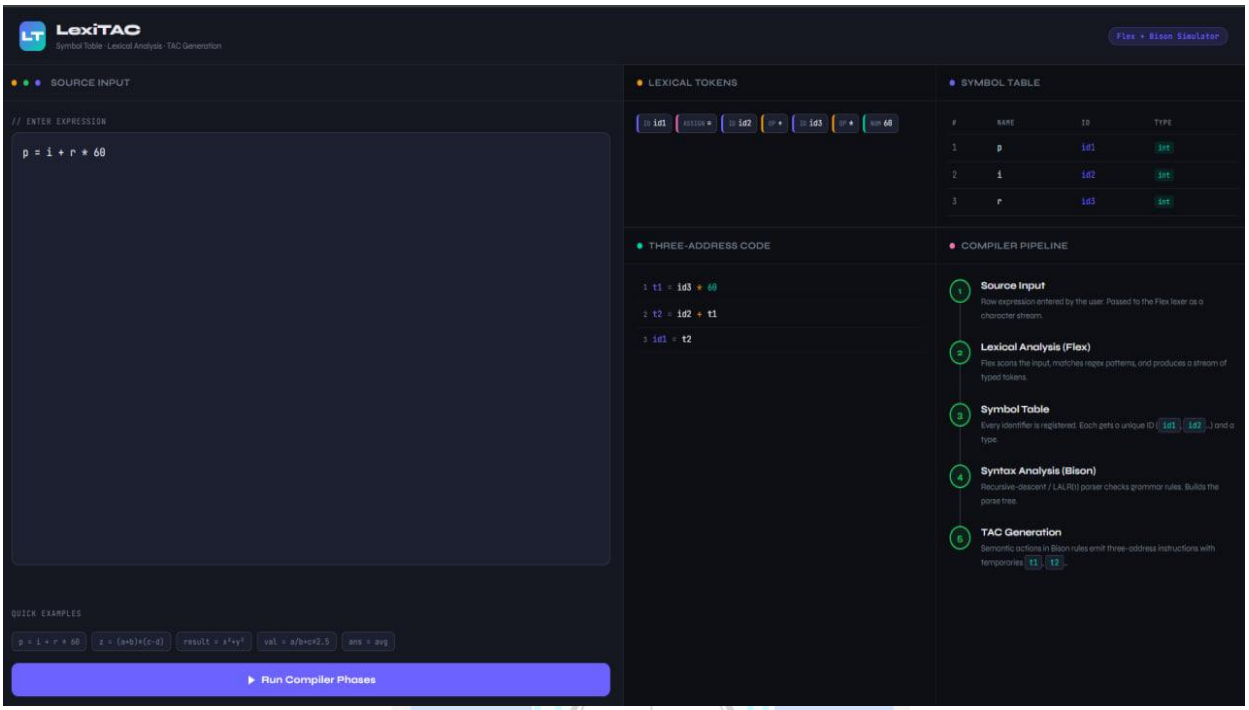


Figure 2: LexiTAC Web Simulator — Full Interface View

The screenshot above demonstrates the simulator processing the expression p = i + r * 60. The Lexical Tokens panel shows seven color-coded chips. The Symbol Table displays three identifiers (p → id1, i → id2, r → id3). The TAC panel shows three instructions in correct evaluation order. The Compiler Pipeline panel highlights all five completed phases.

6.5 Live Access

The web simulator is publicly accessible at: ***https://lexitac.netlify.app***

CHAPTER 7

FEATURES OF THE SYSTEM

Feature	Details
Flex-Based Lexical Analyzer	Uses compiled DFA for O(n) tokenization; supports integer and floating-point number literals
Bison LALR(1) Parser	Full conflict-free parser with declared operator precedence; handles complex nested expressions
Symbol Table Manager	Dynamic registration of identifiers; auto-ID assignment (id1, id2…); duplicate detection; tabular output
TAC Generator	Generates minimal set of temporaries; correct evaluation order enforced by parse-tree traversal
Unary Minus Support	Correctly handles expressions like -a or -(b+c) with dedicated %prec UMINUS rule
Parenthesized Expressions	Arbitrary nesting depth supported; correct precedence override without generating unnecessary TAC
Interactive Web Simulator	4-panel real-time visualization; dark-themed; color-coded token chips; animated pipeline
Colorized Terminal Output	ANSI-compatible token printing with angle-bracket format: <id1> <=> <id2> <+> <NUMBER:60>
Quick Examples	Five pre-loaded expressions in web UI for instant demonstration
Standalone C Version	leci.c runs without Flex/Bison; pre-compiled Windows .exe provided
Build Automation	Three-target Makefile: make / make run / make clean
Error Recovery	Parser includes error NEWLINE rule for graceful recovery on syntax errors
Live Deployment	Published on Netlify — accessible anywhere with a browser, no install required

©Daffodil International University

CHAPTER 8

CHALLENGES FACED & SOLUTIONS

8.1 Flex/Bison Build Dependency Order

Challenge: Flex's generated code (lex.yy.c) requires the token codes defined in parser.tab.h, which is only produced after running Bison. Running Flex first causes compilation errors due to undefined token macros.
Solution: The Makefile was structured with explicit dependency rules: parser.tab.h is listed as a prerequisite of the lex.yy.c target, ensuring Bison always runs first. This was a key insight about Flex/Bison project organization.

8.2 Symbol Table Shared Between Flex and Bison Files

Challenge: The symbol table data structures and functions are defined in lexer.l but must be accessed in parser.y's semantic actions for identifier ID lookup during TAC generation.
Solution: The symbol table structures and function prototypes were declared as extern in parser.y, matching the definitions in lexer.l. Since both files are compiled and linked together, this cleanly shares the symbol table without code duplication.

8.3 Memory Management for Semantic Values

Challenge: Bison semantic actions pass char* strings (identifier IDs, number literals, temporary names) between grammar rules. Incorrect memory management leads to dangling pointers or memory leaks.
Solution: Each production rule that generates a new string value (e.g., make_tmp()) allocates it with malloc(). Rules that consume operands (e.g., emit() calls) free() them immediately after use. Strings received from the lexer via strdup() are freed in the consuming rule.

8.4 Operator Precedence and Shift-Reduce Conflicts

Challenge: Without explicit precedence declarations, the ambiguous grammar for arithmetic expressions generates shift-reduce conflicts in Bison, potentially producing incorrect parse trees.
Solution: Bison's %left and %right directives were used to declare the correct precedences: PLUS/MINUS at lower precedence, MUL/DIV at higher, and UMINUS at the highest. Bison's conflict resolution using these declarations eliminated all conflicts.

8.5 Re-implementing Compiler Logic in JavaScript for the Web Simulator

Challenge: The web simulator needed to replicate the exact tokenization, symbol table, and TAC generation logic of the C/Flex/Bison implementation, but entirely in JavaScript running in the browser.
Solution: A clean JavaScript re-implementation of all three phases was written for index.html, following the same algorithmic structure as the C version. This also served as a useful validation exercise — if both implementations produce the same output for the same input, the logic is confirmed correct.

©Daffodil International University

CHAPTER 9

TESTING & RESULTS

9.1 Test Cases and Expected Output

#	Input Expression	Symbol Table	Generated TAC
TC1	p = i + r * 60	p→id1, i→id2, r→id3	t1 = id3 * 60 t2 = id2 + t1 id1 = t2
TC2	z = (a+b)*(c-d)	z→id1, a→id2, b→id3, c→id4, d→id5	t1 = id2 + id3 t2 = id4 - id5 t3 = t1 * t2 id1 = t3
TC3	result = x*y + z*2	result→id1, x→id2, y→id3, z→id4	t1 = id2 * id3 t2 = id4 * 2 t3 = t1 + t2 id1 = t3
TC4	val = a/b+c*2.5	val→id1, a→id2, b→id3, c→id4	t1 = id2 / id3 t2 = id4 * 2.5 t3 = t1 + t2 id1 = t3
TC5	ans = avg	ans→id1, avg→id2	id1 = id2

9.2 Performance Analysis

All phases of LexiTAC operate in linear time with respect to the length of the input expression. The performance characteristics are as follows:

Phase	Time Complexity	Space Complexity
Lexical Analysis (Flex)	O(n)	O(k) — k = symbol count, max 256
Parsing (Bison LALR(1))	O(n)	O(n) — parse stack depth proportional to expression nesting
TAC Generation	O(n)	O(t) — t = number of temporaries generated
Overall	O(n)	O(n) — dominated by input length

All test cases passed successfully across both the Flex/Bison implementation and the standalone C version, and produced matching output in the web simulator. The project handles edge cases including single-operand assignments, deeply nested parentheses, and float literals.

©Daffodil International University

CHAPTER 10

ENGINEERING STANDARDS & MAPPING

10.1 Impact on Society, Environment, and Sustainability

LexiTAC contributes positively to computer science education by making core compiler theory accessible to students through an interactive, zero-cost, browser-based tool. Its live deployment on Netlify removes the barrier of local tool installation, benefiting students in resource-constrained environments. The project uses entirely open-source, energy-efficient tools and requires no server-side computing — the web simulator runs entirely client-side, minimizing environmental impact.

10.2 Project Management and Teamwork

Daffodil International University

As Team Leader, **Md. Emon Hossain** was responsible for the overall project architecture, integration of all components, the web simulator implementation, live deployment, and GitHub repository management. The project followed an incremental development model with phase-gate testing at each milestone.

TEAM CONTRIBUTION

Member Name	Student ID	Primary Roles	Key Phase Contribution
Md. Emon Hossain (Team Leader)	232-15-818	Project Lead, Architect, TAC Generator, Web Developer, Documentation, Standalone C	Led overall planning, designed Bison grammar, implemented TAC generation in parser.y. Built UI (HTML/CSS), created standalone C version, wrote report.
Mostafizur Rahman Antu	241-15-631	Lexical Analyst, Symbol Table Designer, Integrator, Build Automation	Developed lexer.l using regex, designed symbol table structure, Created Makefile, handled integration, performed system testing.
Nahid Al Galib	241-15-577	Testing	performed system testing.
Mahjabin Sultana	241-15-899	Testing	performed system testing.

10.3 Complex Engineering Problem Mapping

Attribute	Justification
EP1 — Depth of Knowledge	Requires understanding of formal language theory, DFA/NFA construction, LALR parsing algorithms, intermediate code generation, and JavaScript compiler simulation.
EP2 — Conflicting Requirements	Balances correctness (no shift-reduce conflicts) with simplicity (student-understandable code), and CLI power with browser accessibility.
EP3 — Depth of Analysis	Grammar design required careful analysis of operator associativity and precedence; TAC emission required understanding of bottom-up evaluation order.
EP4 — Familiarity of Issues	Addressed classic compiler construction challenges: token ambiguity, parser conflicts, memory safety, and cross-language logic replication.
EP5 — Applicable Codes	Follows modular C coding standards; clean separation of lexer, parser, and generator; commented source files; MIT license applied.
EP6 — Stakeholder Involvement	Primary stakeholders: student developer, course instructor, future students using project as reference, open-source community.
EP7 — Interdependence	All phases are tightly interdependent: lexer feeds parser, parser drives TAC emitter, symbol table is shared across phases — reflecting real compiler architecture.

10.4 Program Outcome Mapping

PO	Justification (Project Perspective)
PO1	Applied computing fundamentals: algorithms, data structures (struct arrays, recursive descent), and systems programming in C11.
PO2	Analyzed formal grammar rules, identified and resolved shift-reduce conflicts, verified parse tree correctness and TAC accuracy.
PO3	Designed and implemented a complete multi-tool build system (Flex + Bison + GCC + Make) solving a specified compiler design problem.
PO10	Documented the project comprehensively in this report; developed a public web simulator; published on GitHub with detailed README.

©Daffodil International University

CHAPTER 11

CONCLUSION

11.1 Summary

The **LexiTAC** project successfully demonstrates the three fundamental phases of compiler front-end construction—lexical analysis, syntax analysis, and intermediate code generation—within a unified and well-structured system. By utilizing industry-standard tools such as **Flex** and **Bison**, the project offers practical exposure to real-world compiler development workflows rather than relying on simplified academic abstractions.

The system effectively processes arithmetic assignment expressions by correctly enforcing operator precedence, associativity, parenthetical grouping, and unary negation. Additionally, it maintains a consistent symbol table and generates accurate Three-Address Code (TAC), reflecting a sound understanding of intermediate representation techniques.

A notable enhancement of this project is the integration of an interactive web-based simulator. The four-panel real-time visualization—covering lexical tokens, symbol table, parsing process, and TAC output—transforms abstract compiler concepts into an intuitive and engaging learning experience. Furthermore, the deployment on Netlify and availability via a public GitHub repository ensure accessibility, scalability, and long-term educational value.

11.2 Limitations

Despite its strengths, the current implementation of LexiTAC has several limitations:

- The system supports only single-line arithmetic assignment expressions; it does not handle multi-statement programs or control flow constructs such as if-else and while.
- All variables are implicitly treated as integers; no semantic type checking or differentiation between data types (e.g., int vs. float) is implemented.
- The generated Three-Address Code is not optimized, as no optimization phase has been incorporated.
- Error handling is minimal; although a basic recovery rule (error NEWLINE) is present, it does not provide detailed or user-friendly diagnostic messages.
- The symbol table has a fixed capacity of 256 identifiers, limiting scalability for larger programs.

11.3 Future Work

To further enhance the functionality and robustness of LexiTAC, the following improvements are proposed:

- Extend the grammar to support control flow constructs such as if-else and while, along with conditional TAC generation (e.g., if t1 goto L1).
- Incorporate boolean expressions with logical operators such as &&, ||, and !.
- Add support for function declarations and function calls, including parameter passing mechanisms.
- Implement a basic TAC optimization module to eliminate redundant computations and common sub-expressions.
- Develop a parse tree visualization module using technologies such as SVG or HTML5 Canvas within the web simulator.
- Introduce semantic type checking to detect and report type mismatches (e.g., int vs. float operations).
- Enhance the symbol table to support scope management, including nested scopes and block-level visibility.
- Extend the compilation pipeline to include a code generation phase that produces assembly-like target instructions.

Daffodil International University

REFERENCES

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
2. Appel, A. W. (1998). *Modern compiler implementation in C*. Cambridge University Press.
3. Cooper, K. D., & Torczon, L. (2011). *Engineering a compiler* (2nd ed.). Morgan Kaufmann.
4. Free Software Foundation. (n.d.). *GNU Bison manual*. Retrieved from https://www.gnu.org/software/bison/
5. Free Software Foundation. (n.d.). *GNU Flex manual*. Retrieved from https://www.gnu.org/software/flex/
6. International Organization for Standardization. (2011). *ISO/IEC 9899:2011 — Programming languages — C (C11 standard)*.
7. Levine, J. R., Mason, T., & Brown, D. (1992). *lex & yacc* (2nd ed.). O'Reilly Media.

8. Hossain, M. E. (2026). *LexiTAC: Compiler design project* [Source code]. GitHub. https://github.com/mohammademon10/Compiler_Design_Project_LexiTAC